

Ejercicio 4 - Funciones que reciben y retornan funciones

1. Define una función que recibe una función que transforma un dato (por ejemplo, conversión de grados Celsius a Fahrenheit), y lo aplica a una lista de datos.

```
2. procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
3. #Implementa procesarLecturas
4.
5. Ejemplo:
6. procesarLecturas celsiusAFahrenheit [0,12,20]
```

7. Define una función que retorne un filtro personalizado:

```
8. crearFiltro :: (a -> Bool) -> ([a] -> [a])
9. #Implementa crearFiltro
10.
11. Ejemplo:
12.
13. soloPares = crearFiltro even
14. soloPares [1..10] == [2,4,6,8,10]
```

15. Crea una función que genera "verificadores" de rangos.

```
16. enRango :: Int -> Int -> (Int -> Bool)
17. #Implementa enRango
18.
19. Ejemplo:
20.
21. entre5y10 = enRango 5 10
22. entre5y10 7 -- True
```

23. Genera una función que eleva a una potencia dada.

```
24. potenciador :: Int -> (Int -> Int)
25. #Implementa potenciador
26.
27. Ejemplo:
28.
29. potencia5 = potenciador 5
30. potencia5 7
```

Solucion

```
procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
procesarLecturas f xs = map f xs

celsiusAFahrenheit :: Double -> Double
celsiusAFahrenheit c = c * 9 / 5 + 32

crearFiltro :: (a -> Bool) -> ([a] -> [a])
crearFiltro condicion = filter condicion

soloPares :: [Int] -> [Int]
soloPares = crearFiltro even

enRango :: Int -> Int -> (Int -> Bool)
enRango minimo maximo = \x -> x >= minimo && x <= maximo

entre5y10 :: Int -> Bool
entre5y10 = enRango 5 10

potenciador :: Int -> (Int -> Int)
potenciador n = \x -> x ^ n

potencia5 :: Int -> Int
potencia5 = potenciador 5
```

```
: procesarLecturas celsiusAFahrenheit [0,12,20]
soloPares [1..10]
entre5y10 7
entre5y10 11
potencia5 7

[32.0,53.6,68.0]
[2,4,6,8,10]
True
False
16807
```

Ejercicio 5 - “El problema de las dos jarras” usando A*

Objetivo:

Usar el algoritmo A* para encontrar la secuencia más corta de acciones que permita medir exactamente 4 litros usando dos jarras:

- una de 5 litros,
- otra de 3 litros,
- y una fuente de agua infinita.

Descripción del problema

Tienes:

- Jarra A de capacidad 5 L
- Jarra B de capacidad 3 L

Operaciones posibles:

- Llenar una jarra completamente.
- Vaciar una jarra.
- Verter el contenido de una jarra en la otra hasta que una se vacíe o la otra se llene.

Estado inicial: (0, 0)

Meta: Estado donde la primera jarra tenga 4 litros, es decir: (4, 0)

Representación de estados

Cada estado es un par (a, b) donde:

- a = litros en la jarra de 5 L
- b = litros en la jarra de 3 L

```
type Estado = (Int, Int)
type Nodo = Estado
type Costo = Int
type Grafo = [(Nodo, [(Nodo, Costo)])]
```

Definimos las operaciones válidas desde un estado (a, b):

```
capA, capB :: Int
capA = 5
capB = 3

-- Genera todos los estados alcanzables desde uno dado
vecinos :: Estado -> [(Estado, Costo)]
vecinos (a,b) =
  [ ((capA, b), 1)           -- Llenar jarra A
  , ((a, capB), 1)          -- Llenar jarra B
  , ((0, b), 1)             -- Vaciar jarra A
  , ((a, 0), 1)             -- Vaciar jarra B
  , (transferirAB (a,b), 1)  -- Verter A→B
  , (transferirBA (a,b), 1)  -- Verter B→A
  ]
```

Definir funciones de transición de estados

```
-- Verter de A a B
-- Definir transferirAB :: Estado -> Estado
```

```
-- Verter de B a A
-- Definir transferirBA :: Estado -> Estado
```

Definir una heurística admisible

```
heuristica :: Estado -> Costo
```

Probar la solución con el algoritmo base de A*

```
import Data.List (sortOn)

aStar :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> [(Nodo, Costo)]
aStar sucesores heuristica inicio meta = buscar [(inicio, 0)] []
  where
    buscar [] _ = []
    buscar ((nodo, costo):cola) visitados
      | nodo == meta = [(nodo, costo)]
      | nodo `elem` visitados = buscar cola visitados
      | otherwise =
        let nuevos = [ (v, costo + c) | (v, c) <- sucesores nodo, v `notElem` visitados ]
        ordenados = sortOn (\(v, c) -> c + heuristica v) (cola ++ nuevos)
        in (nodo, costo) : buscar ordenados (nodo : visitados)

aStar vecinos heuristica inicio meta
```

Solucion

```
import Data.List (sortOn)

type Estado = (Int, Int)
type Nodo = Estado
type Costo = Int
type Grafo = [(Nodo, [(Nodo, Costo)])]

capA, capB :: Int
capA = 5
capB = 3

vecinos :: Estado -> [(Estado, Costo)]
vecinos (a,b) =
  [ ((capA, b), 1)
  , ((a, capB), 1)
  , ((0, b), 1)
  , ((a, 0), 1)
  , (transferirAB (a,b), 1)
  , (transferirBA (a,b), 1)
  ]

transferirAB :: Estado -> Estado
transferirAB (a, b) = (a - cantidad, b + cantidad)
  where cantidad = min a (capB - b)

transferirBA :: Estado -> Estado
transferirBA (a, b) = (a + cantidad, b - cantidad)
  where cantidad = min b (capA - a)

heuristica :: Estado -> Costo
heuristica (a, b) = if (a, b) == (4, 0) then 0 else 1

aStar :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> [(Nodo, Costo)]
aStar sucesores heur inicio meta = buscar [(inicio, 0)] []
  where
    buscar [] _ = []
    buscar ((nodo, costo):cola) visitados
      | nodo == meta = [(nodo, costo)]
      | nodo `elem` visitados = buscar cola visitados
      | otherwise =
        let nuevos = [ (v, costo + c) | (v, c) <- sucesores nodo, v `notElem` visitados ]
            ordenados = sortOn (\(v, c) -> c + heur v) (cola ++ nuevos)
        in (nodo, costo) : buscar ordenados (nodo : visitados)

19]: transferirAB :: Estado -> Estado
transferirAB (a, b) = (a - cantidad, b + cantidad)
  where cantidad = min a (capB - b)

transferirBA :: Estado -> Estado
transferirBA (a, b) = (a + cantidad, b - cantidad)
  where cantidad = min b (capA - a)

-- 0 si ya es la meta, 1 si no (nunca sobreestima)
heuristica :: Estado -> Costo
heuristica (a, b) = if (a, b) == (4, 0) then 0 else 1

20]: transferirAB (5,0)
transferirBA (2,3)
aStar vecinos heuristica (0,0) (4,0)

(2,3)
(5,0)
[(0,0),0],((5,0),1),((0,3),1),((5,3),2),((2,3),2),((3,0),2),((2,0),3),((3,3),3),((0,2),4),((5,1),4),((5,2),5),((0,1),5),((4,3),6),((1,0),6),((4,0),7)]
```

Retrospectiva

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?
(Horas/Hombre)

R: El tiempo invertido fue de 5 horas porque tuvimos que hacer la retroalimentación de lo que estábamos viendo y poder comprender bien lo que escribimos

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

R: el laboratorio ya se encuentra totalmente terminado

3. ¿Cuál consideran fue el mayor logro? ¿Por qué?

R: El mayor logro fue entender bien el código que estábamos escribiendo y porque estaba de esa manera

4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

R: El mayor problema técnico por lo menos en la primera parte fueron los computadores del laboratorio están un poco desactualizados y es un poco complicado el manejo del mouse y lo lento que cargan

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

R: nos comunicamos bien y nos encontramos para terminar de desarrollar el laboratorio, lo cual ayudó a entender bien el tema y aportar para terminarlo más rápido

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

R: usamos las presentaciones que están en moodle y algunos recursos más de la web También inteligencia artificial como Claude y Gemini para poder entender bien el código